

Introduction to SELECT

SELECT retrieves data from one or more tables.

Syntax:

```
SELECT column1, column2 FROM table_name;
```

Example 1

```
SELECT first_name, last_name FROM  
employees;
```

Example 2

```
SELECT * FROM departments;
```

Retrieving Data from Tables

Retrieve data from tables.

Example 1:

```
SELECT title, genre, ratingFROM  
Movie;
```

Example 2:

```
SELECT name, capacityFROM Screen;
```

Example 1:

```
SELECT name, price FROM products;
```

Example 2 (Multiple tables using JOIN):

```
SELECT orders.order_id, customers.customer_nameFROM ordersJOIN customers ON  
orders.customer_id = customers.customer_id;
```

Advanced Table Joins

Movie and Show Join

```
SELECT Movie.title,  
Show.show_datetimeFROM MovieJOIN  
Show ON Movie.movie_id =  
Show.movie_id;
```

Food Item and Size Join

```
SELECT FoodItem.name,  
FoodItemSize.size_name,  
FoodItemSize.rateFROM FoodItemJOIN  
FoodItemSize ON FoodItem.item_id =  
FoodItemSize.item_id;
```

Column Aliases

Syntax:

```
SELECT column_name AS alias_name FROM table_name;
```

Rename columns for readability.

01

Basic Alias Example

```
SELECT first_name AS "First Name",  
last_name AS "Last Name" FROM  
employees;
```

03

Movie Title Alias

```
SELECT title AS "Movie Title", rating  
AS "IMDb Rating"FROM Movie;
```

02

Calculated Field Alias

```
SELECT salary * 12 AS "Annual Salary"  
FROM employees;
```

04

Screen Name Alias

```
SELECT name AS "Screen Name", capacity  
AS "Total Seats"FROM Screen;
```


WHERE Clause

Filters rows that satisfy specific conditions.

Filter rows using conditions.

Basic Filtering

```
SELECT * FROM students WHERE grade = 'A';
```

```
SELECT * FROM employees WHERE salary  
> 50000;
```

Movie Database Examples

```
SELECT title, statusFROM MovieWHERE  
status = 'Now Showing';
```

```
SELECT name, capacityFROM ScreenWHERE  
capacity > 150;
```


Pattern Matching with LIKE

Used for pattern matching.

Use patterns with % and _.

Logical Operators and Advanced

Logical Operators (AND, OR, NOT) Combine multiple

1

Starts With Pattern

```
SELECT name FROM customers WHERE  
name LIKE 'A%';
```

```
SELECT titleFROM MovieWHERE title  
LIKE 'The%';
```

2

Contains Pattern

```
SELECT email FROM users WHERE email  
LIKE '%@gmail.com';
```

```
SELECT name, emailFROM UserWHERE  
email LIKE '%@gmail.com';
```



Logical Operators and Advanced Filtering

Logical Operators (AND, OR, NOT)

Combine multiple conditions.

Combine conditions logically.

7

AND Operator

```
SELECT * FROM employees WHERE  
department = 'Sales' AND salary >  
50000;
```

```
SELECT title, genre, ratingFROM  
MovieWHERE genre = 'Action' AND  
rating >= 8.0;
```

⊗

NOT Operator

```
SELECT * FROM employees WHERE NOT  
department = 'HR';
```

```
SELECT titleFROM MovieWHERE NOT  
status = 'Released';
```

IN and BETWEEN Operators

IN Operator

Match against multiple possible values.

Match multiple values easily.

IN Examples

```
SELECT * FROM employees WHERE  
department IN ('IT', 'Finance',  
'HR');
```

```
SELECT * FROM products WHERE  
category_id IN (1, 3, 5);
```

```
SELECT title, genreFROM MovieWHERE  
genre IN ('Drama', 'Comedy',  
'Thriller');
```

```
SELECT payment_id, statusFROM  
PaymentWHERE status IN ('Failed',  
'Pending');
```

BETWEEN Operator

Selects values within a range.

Select values within a range.

```
SELECT * FROM products WHERE price  
BETWEEN 100 AND 500;
```

```
SELECT * FROM employees WHERE  
hire_date BETWEEN '2022-01-01' AND  
'2023-01-01';
```

```
SELECT show_id, show_datetimeFROM  
ShowWHERE show_datetime BETWEEN  
'2025-10-07' AND '2025-10-14';
```

```
SELECT title, ratingFROM MovieWHERE  
rating BETWEEN 6.0 AND 8.0;
```


NULL Handling and Data Organization

IS NULL / IS NOT NULL

Checks for missing (null) values.

Check for missing values.

NULL Examples

```
SELECT * FROM students WHERE phone IS NULL;
```

```
SELECT titleFROM MovieWHERE poster_image_url IS NULL;
```

NOT NULL Examples

```
SELECT * FROM students WHERE email IS NOT NULL;
```

```
SELECT payment_id, failure_reasonFROM PaymentWHERE failure_reason IS NOT NULL;
```

GROUP BY / ORDER BY

GROUP BY groups rows for aggregate functions.

ORDER BY sorts the result.

Group and sort results.

GROUP BY Examples

```
SELECT department, AVG(salary) AS avg_salary FROM employees GROUP BY department;
```

```
SELECT genre, AVG(rating) AS avg_ratingFROM MovieGROUP BY genre;
```

ORDER BY Examples

```
SELECT name, salary FROM employees ORDER BY salary DESC;
```

```
SELECT name, rateFROM FoodItemSizeORDER BY rate DESC;
```

Quick Recap

Clause	Purpose	Example
SELECT	Retrieve data	SELECT * FROM table;
WHERE	Filter rows	WHERE salary > 50000;
LIKE	Pattern match	LIKE 'A%'
IN	Match a list	IN ('IT','HR')
BETWEEN	Range filter	BETWEEN 100 AND 500
IS NULL	Null check	IS NOT NULL
GROUP BY	Aggregate grouping	GROUP BY dept
ORDER BY	Sorting	ORDER BY name ASC